

RAJALAKSHMI ENGINEERING COLLEGE
RAJALAKSHMI NAGAR, THANDALAM – 602 105



CS23632 Cryptography and Network Security

Laboratory Record Note Book

Name : S SAI ARAVIND

Year / Branch / Section : 3rd Year - CSE - E.

University Register No. : 2116230701275

College Roll No. : 230701275

Semester : VI Semester

Academic Year : 2025-26

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS23632 Cryptography and Network Security

NAME	SAI ARAVIND S
ROLL NO	230701275
YEAR	3rd Year - CSE
SEC	E



RAJALAKSHMI ENGINEERING COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name: **SAI ARAVIND S**

Academic Year: 2025-25 **Semester:** VI **Branch:** CSE

Register No. 2116230701275

*Certified that this is the bonafide record of work done by the above student in
the Cryptography and Network Security Laboratory during the academic year
2026 – 2027.*

Signature of Faculty in-charge

Submitted for the Practical Examination held on

Internal Examiner

External Examiner

INDEX

Sl NO	Experiment	Date	GitHub
1	Installation and Configuration of Kali Linux/Parrot OS in a VMware/VirtualBox.		
2	Encryption Crypto 101 in TryHackMe Platform		
3	Perform Man-in-the-middle (MITM) attacks using the Ettercap tool.		
4	Demonstrate hash cracking using John the Ripper tool.		
5	Perform various configurations of Iptables Firewall in Linux.		
6	Snort IDS/IPS to detect and prevent real time threats in TryHackMe Platform.		
7	Perform Code Injection on Application Process using Ptrace.		
8	Privilege Escalation in TryHackMe Platform		
9	Demonstrate various exploits of Window OS using Metasploit Framework		
10	Perform Wireless Audit on routers and decrypt the WPA keys using Aircrack-ng		
11	Perform IP Spoofing using a GitHub Tool (Educational Demonstration)		
12	Study of Camera Phishing using Open-Source GitHub Tool		
13	Study of Phishing Attacks using Z-Phisher (GitHub Tool)		
14	Study and Demonstration of Brute Force Password Attack in a Controlled Kali Linux Environment		

EXPERIMENT – 1

Installation and Configuration of Kali Linux in Oracle VirtualBox

Step 1: Install VirtualBox

Download and install Oracle VirtualBox on your system.

Step 2: Download Kali Linux

Download the ISO file of Kali Linux (64-bit) from the official website.

Step 3: Create Virtual Machine

Open VirtualBox and click “New”.

Set:

- Name: Kali Linux
- Type: Linux
- Version: Debian (64-bit)

Step 4: Allocate Resources

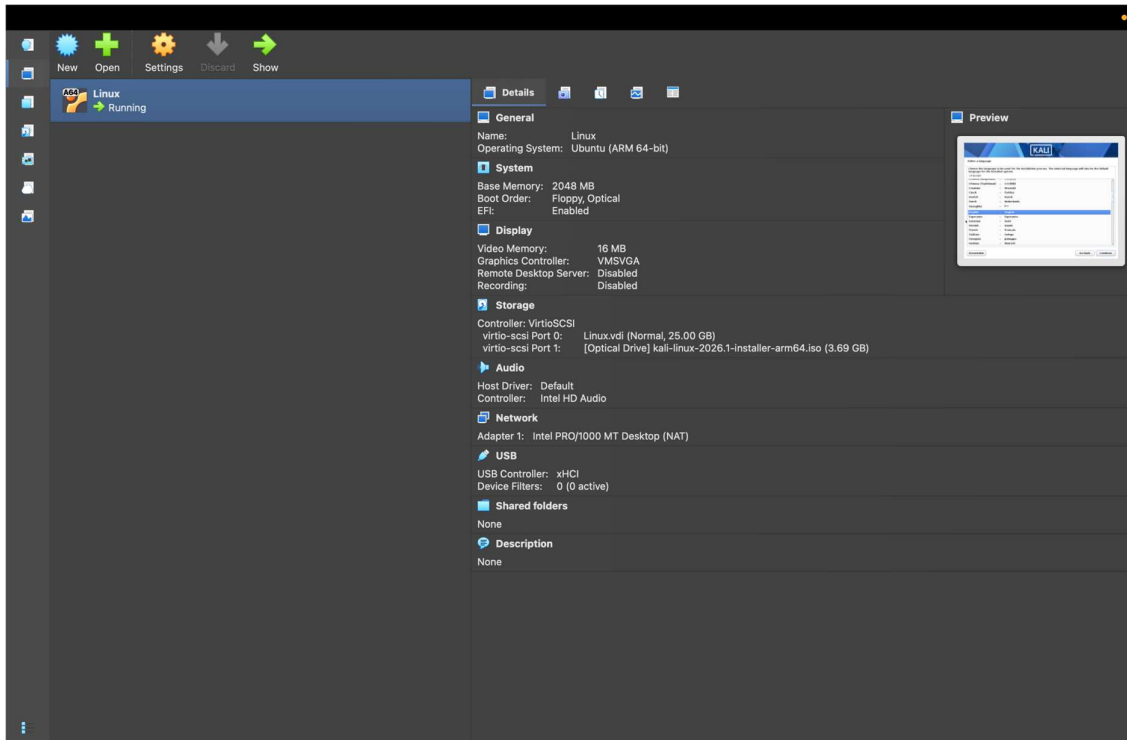
Assign:

1. RAM: Minimum 2 GB
2. CPU: 2 cores
3. Disk size: 30 GB (VDI, dynamically allocated)

Step 5: Install Kali Linux

Attach the ISO file and start the virtual machine.

Select “Graphical Install” and complete the setup by configuring language, user credentials, and disk partition. Reboot after installation.





Result

Kali Linux was successfully installed and configured in Oracle VirtualBox.

EXPERIMENT – 2

Encryption – Crypto 101 using TryHackMe

Step 1: Login to Platform

Open browser and go to TryHackMe.
Login using your account.

Step 2: Open Crypto 101 Room

Search for **Crypto 101** and click **Join Room**.

Step 3: Start Environment

Click **Start Machine / AttackBox** to launch the virtual lab.

Step 4: Perform Cryptography Tasks

- (a) Base64 Encoding/Decoding
- (b) Hex Conversion
- (c) Caesar Cipher

Step 5: Submit Answers

Enter the decoded results in the answer boxes and complete all tasks.

TryHackMe

DashboardLearnPracticeCompete

Go Premium

S

← Back to all walkthroughs



Encryption - Crypto 101

An introduction to encryption, as part of a series on crypto

45 min140,459

Share your achievement

Start AttackBox

Save Room

3935 Recommend

Options

Room completed (100%)

Task 1 What will this room cover?

Task 2 Key terms

Task 3 Why is Encryption important?

Task 4 Crucial Crypto Maths

Task 5 Types of Encryption

Task 6 RSA - Rivest Shamir Adleman

Task 7 Establishing Keys Using Asymmetric Cryptography

Task 8 Digital signatures and Certificates

Task 9 SSH Authentication



TryHackMe

DashboardLearnPracticeCompete

Go Premium

S

← Back to all walkthroughs



Encryption - Crypto 101

An introduction to encryption, as part of a series on crypto

45 min140,459

Share your achievement

Start AttackBox

Save Room

3935 Recommend

Options

Room completed (100%)

Task 1 What will this room cover?

This room will cover:

- Why cryptography matters for security and CTFs
- The two main classes of cryptography and their uses
- RSA, and some of the uses of RSA
- 2 methods of Key Exchange
- Notes about the future of encryption with the rise of Quantum Computing

Note: This room expects some familiarity with tools, and some research into how to use them yourself!

Answer the questions below

I'm ready to learn about encryption

No answer neededCorrect Answer

Task 2 Key terms

tryhackme.com

Room completed (100%)

Task 2

Key terms

Many of these key terms are shared with <https://tryhackme.com/room/hashingcrypto101>, so you might be able to skip over some if you're already familiar.

Ciphertext - The result of encrypting a plaintext, encrypted data

Cipher - A method of encrypting or decrypting data. Modern ciphers are cryptographic, but there are many non cryptographic ciphers like Caesar.

Plaintext - Data before encryption, often text but not always. Could be a photograph or other file

Encryption - Transforming data into ciphertext, using a cipher.

Encoding - NOT a form of encryption, just a form of data representation like base64. Immediately reversible.

Key - Some information that is needed to correctly decrypt the ciphertext and obtain the plaintext.

Passphrase - Separate to the key, a passphrase is similar to a password and used to protect a key.

Asymmetric encryption - Uses different keys to encrypt and decrypt.

Symmetric encryption - Uses the same key to encrypt and decrypt

Brute force - Attacking cryptography by trying every different password or every different key

Cryptanalysis - Attacking cryptography by finding a weakness in the underlying maths

Alice and Bob - Used to represent 2 people who generally want to communicate. They're named Alice and Bob because this gives them the initials A and B.
https://en.wikipedia.org/wiki/Alice_and_Bob for more information, as these extend through the alphabet to represent many different people involved in communication.

WARNING: This room is very theory heavy. Cryptography is a big topic, and this room is designed to just scratch the surface.

Answer the questions below

I agree not to complain too much about how theory heavy this room is.

No answer needed

✓ Correct Answer

Are SSH keys protected with a passphrase or a password?

passphrase

✓ Correct Answer

\$

tryhackme.com

Room completed (100%)

Task 2

Key terms

Task 3

Why is Encryption important?

Cryptography is used to protect confidentiality, ensure integrity, ensure authenticity. You use cryptography every day most likely, and you're almost certainly reading this now over an encrypted connection.

When logging into TryHackMe, your credentials were sent to the server. These were encrypted, otherwise someone would be able to capture them by snooping on your connection.

When you connect to SSH, your client and the server establish an encrypted tunnel so that no one can snoop on your session.

When you connect to your bank, there's a certificate that uses cryptography to prove that it is actually your bank rather than a hacker.

When you download a file, how do you check if it downloaded right? You can use cryptography here to verify a checksum of the data.

You rarely have to interact directly with cryptography, but it silently protects almost everything you do digitally.

Whenever sensitive user data needs to be stored, it should be encrypted. Standards like PCI-DSS state that the data should be encrypted both at rest (in storage) AND while being transmitted. If you're handling payment card details, you need to comply with these PCI regulations. Medical data has similar standards. With legislation like GDPR and California's data protection, data breaches are extremely costly and dangerous to you as either a consumer or a business.

DO NOT encrypt passwords unless you're doing something like a password manager. Passwords should not be stored in plaintext, and you should use hashing to manage them safely.

Answer the questions below

What does SSH stand for?

Secure Shell

✓ Correct Answer

How do web servers prove their identity?

certificates

✓ Correct Answer

\$

What is the main set of standards you need to comply with if you store or process payment card details?

PCI-DSS

✓ Correct Answer

Task 4

Crucial Crypto Maths

tryhackme.com

Room completed (100%)

Task 3 Why is Encryption important?

Task 4 Crucial Crypto Maths

Task 5 Types of Encryption

Task 6 RSA - Rivest Shamir Adleman

Task 7 Establishing Keys Using Asymmetric Cryptography

There's a little bit of math(s) that comes up relatively often in cryptography. The Modulo operator. Pretty much every programming language implements this operator, or has it available through a library. When you need to work with large numbers, use a programming language. Python is good for this as integers are unlimited in size, and you can easily get an interpreter.

When learning division for the first time, you were probably taught to use remainders in your answer. $X \% Y$ is the remainder when X is divided by Y .

Examples

$25 \% 5 = 0$ ($5 * 5 = 25$ so it divides exactly with no remainder)

$23 \% 6 = 5$ (23 does not divide evenly by 6 , there would be a remainder of 5)

An important thing to remember about modulo is that it's not reversible. If I gave you an equation: $x \% 5 = 4$, there are infinite values of x that will be valid.

Answer the questions below

What's $30 \% 5$?

0 ✓ Correct Answer

What's $25 \% 7$?

4 ✓ Correct Answer

What's $118613842 \% 9091$?

3565 ✓ Correct Answer

tryhackme.com

Room completed (100%)

Task 4 Crucial Crypto Maths

Task 5 Types of Encryption

Task 6 RSA - Rivest Shamir Adleman

Task 7 Establishing Keys Using Asymmetric Cryptography

Task 8 Digital signatures and Certificates

The two main categories of Encryption are symmetric and asymmetric.

Symmetric encryption uses the same key to encrypt and decrypt the data. Examples of Symmetric encryption are DES (Broken) and AES. These algorithms tend to be faster than asymmetric cryptography, and use smaller keys (128 or 256 bit keys are common for AES, DES keys are 56 bits long).

Asymmetric encryption uses a pair of keys, one to encrypt and the other in the pair to decrypt. Examples are RSA and Elliptic Curve Cryptography. Normally these keys are referred to as a public key and a private key. Data encrypted with the private key can be decrypted with the public key, and vice versa. Your private key needs to be kept private, hence the name. Asymmetric encryption tends to be slower and uses larger keys, for example RSA typically uses 2048 to 4096 bit keys.

RSA and Elliptic Curve cryptography are based around different mathematically difficult (intractable) problems, which give them their strength. More about RSA later.

Answer the questions below

Should you trust DES? Yea/Nay

Nay ✓ Correct Answer

What was the result of the attempt to make DES more secure so that it could be used for longer?

Triple DES ✓ Correct Answer

Is it ok to share your public key? Yea/Nay

Yea ✓ Correct Answer

tryhackme.com

Room completed (100%)

Task 6 RSA - Rivest Shamir Adleman

The math(s) side

RSA is based on the mathematically difficult problem of working out the factors of a large number. It's very quick to multiply two prime numbers together, say $17 \times 23 = 391$, but it's quite difficult to work out what two prime numbers multiply together to make 14351 (113x127 for reference).

The attacking side

The maths behind RSA seems to come up relatively often in CTFs, normally requiring you to calculate variables or break some encryption based on them. The wikipedia page for [RSA](#) seems complicated at first, but will give you almost all of the information you need in order to complete challenges.

There are some excellent tools for defeating RSA challenges in CTFs, and my personal favorite is <https://github.com/Ganapati/RsaCtfTool> which has worked very well for me. I've also had some success with <https://github.com/lus/rsatool>.

The key variables that you need to know about for RSA in CTFs are p , q , m , n , e , d , and c .

" p " and " q " are large prime numbers, " n " is the product of p and q .

The public key is n and e , the private key is n and d .

" m " is used to represent the message (in plaintext) and " c " represents the ciphertext (encrypted text).

CTFs involving RSA

Crypto CTF challenges often present you with a set of these values, and you need to break the encryption and decrypt a message to retrieve the flag.

There's a lot more maths to RSA, and it gets quite complicated fairly quickly. If you want to learn the maths behind it, I recommend reading MuirlandOracle's blog post here: <https://mairlandoracle.co.uk/2020/01/29/rsa-encryption/>.

Answer the questions below

$p = 4391$, $q = 6659$. What is n ?

✓ Correct Answer

I understand enough about RSA to move on, and I know where to look to learn more if I want to.

✓ Correct Answer

tryhackme.com

Room completed (100%)

Task 6 RSA - Rivest Shamir Adleman

Task 7 Establishing Keys Using Asymmetric Cryptography

A very common use of asymmetric cryptography is exchanging keys for symmetric encryption.

Asymmetric encryption tends to be slower, so for things like HTTPS symmetric encryption is better.

But the question is, how do you agree a key with the server without transmitting the key for people snooping to see?

Metaphor time

Imagine you have a secret code, and instructions for how to use the secret code. If you want to send your friend the instructions without anyone else being able to read it, what you could do is ask your friend for a lock.

Only they have the key for this lock, and we'll assume you have an indestructible box that you can lock with it.

If you send the instructions in a locked box to your friend, they can unlock it once it reaches them and read the instructions.

After that, you can communicate in the secret code without risk of people snooping.

In this metaphor, the secret code represents a symmetric encryption key, the lock represents the server's public key, and the key represents the server's private key.

You've only used asymmetric cryptography once, so it's fast, and you can now communicate privately with symmetric encryption.

The Real World

In reality, you need a little more cryptography to verify the person you're talking to is who they say they are, which is done using digital signatures and certificates. You can find a lot more detail on how HTTPS (one example where you need to exchange keys) really works from this excellent blog post. <https://robertheaton.com/2014/03/27/how-does-https-actually-work/>

Answer the questions below

I understand how keys can be established using Public Key (asymmetric) cryptography.

✓ Correct Answer

Task 8 Digital signatures and Certificates

tryhackme.com

Room completed (100%)

Task 8 Digital signatures and Certificates

What's a Digital Signature?

Digital signatures are a way to prove the authenticity of files, to prove who created or modified them. Using asymmetric cryptography, you produce a signature with your private key and it can be verified using your public key. As only you should have access to your private key, this proves you signed the file. Digital signatures and physical signatures have the same value in the UK, legally.

The simplest form of digital signature would be encrypting the document with your private key, and then if someone wanted to verify this signature they would decrypt it with your public key and check if the files match.

Certificates - Prove who you are!

Certificates are also a key use of public key cryptography, linked to digital signatures. A common place where they're used is for HTTPS. How does your web browser know that the server you're talking to is the real tryhackme.com?

The answer is certificates. The web server has a certificate that says it is the real tryhackme.com. The certificates have a chain of trust, starting with a root CA (certificate authority). Root CAs are automatically trusted by your device, OS, or browser from install. Certs below that are trusted because the Root CAs say they trust that organisation. Certificates below that are trusted because the organisation is trusted by the Root CA and so on. There are long chains of trust. Again, this blog post explains this much better than I can.

<https://robertheaton.com/2014/03/27/how-does-https-actually-work/>

You can get your own HTTPS certificates for domains you own using Let's Encrypt for free. If you run a website, it's worth setting it up.

Answer the questions below

What can you use to verify that a file has not been modified and is the authentic file as the author intended?

Digital Signature

✓ Correct Answer

Task 9 SSH Authentication

Task 10 Explaining Diffie Hellman Key Exchange

Task 11 PGP, GPG and AES

tryhackme.com

Room completed (100%)

and never leaves your system.

Using tools like John the Ripper, you can attack an encrypted SSH key to attempt to find the passphrase, which highlights the importance of using a secure passphrase and keeping your private key private.

When generating an SSH key to log in to a remote machine, you should generate the keys on your machine and then copy the public key over as this means the private key never exists on the target machine. For temporary keys generated for access to CTF boxes, this doesn't matter as much.

How do I use these keys?

The `~/.ssh` folder is the default place to store these keys for OpenSSH. The `authorized_keys` (note the US English spelling) file in this directory holds public keys that are allowed to access the server if key authentication is enabled. By default on many distros, key authentication is enabled as it is more secure than using a password to authenticate. Normally for the root user, only key authentication is enabled.

In order to use a private SSH key, the permissions must be set up correctly otherwise your SSH client will ignore the file with a warning. Only the owner should be able to read or write to the private key (600 or stricter). `ssh -i keyNameGoesHere user@host` is how you specify a key for the standard Linux OpenSSH client.

Using SSH keys to get a better shell

SSH keys are an excellent way to "upgrade" a reverse shell, assuming the user has login enabled (www-data normally does not, but regular users and root will). Leaving an SSH key in `authorized_keys` on a box can be a useful backdoor, and you don't need to deal with any of the issues of unbalanced reverse shells like Control-C or lack of tab completion.

Answer the questions below

I recommend giving this a go yourself. Deploy a VM, like Linux Fundamentals 2 and try to add an SSH key and log in with the private key.

No answer needed

✓ Correct Answer

Download the SSH Private Key attached to this room.

No answer needed

✓ Correct Answer

What algorithm does the key use?

RSA

✓ Correct Answer

Crack the password with John The Ripper and rockyou, what's the passphrase for the key?

delicious

✓ Correct Answer

tryhackme.com

Room completed (100%)

Task 9 SSH Authentication

Task 10 Explaining Diffie Hellman Key Exchange

Task 11 PGP, GPG and AES

Task 12 The Future - Quantum Computers and Encryption

What is Key Exchange?

Key exchange allows 2 people/parties to establish a set of common cryptographic keys without an observer being able to get these keys. Generally, to establish common symmetric keys.

How does Diffie Hellman Key Exchange work?

Alice and Bob want to talk securely. They want to establish a common key, so they can use symmetric cryptography, but they don't want to use key exchange with asymmetric cryptography. This is where DH Key Exchange comes in.

Alice and Bob both have secrets that they generate, let's call these A and B. They also have some common material that's public, let's call this C.

We need to make some assumptions. Firstly, whenever we combine secrets/material it's impossible or very very difficult to separate. Secondly, the order that they're combined in doesn't matter.

Alice and Bob will combine their secrets with the common material, and form AC and BC. They will then send these to each other, and combine that with their secrets to form two identical keys, both ABC. Now they can use this key to communicate.

Extra Resources

An excellent video if you want a visual explanation is available here. <https://www.youtube.com/watch?v=NmM9HA2MQGI>

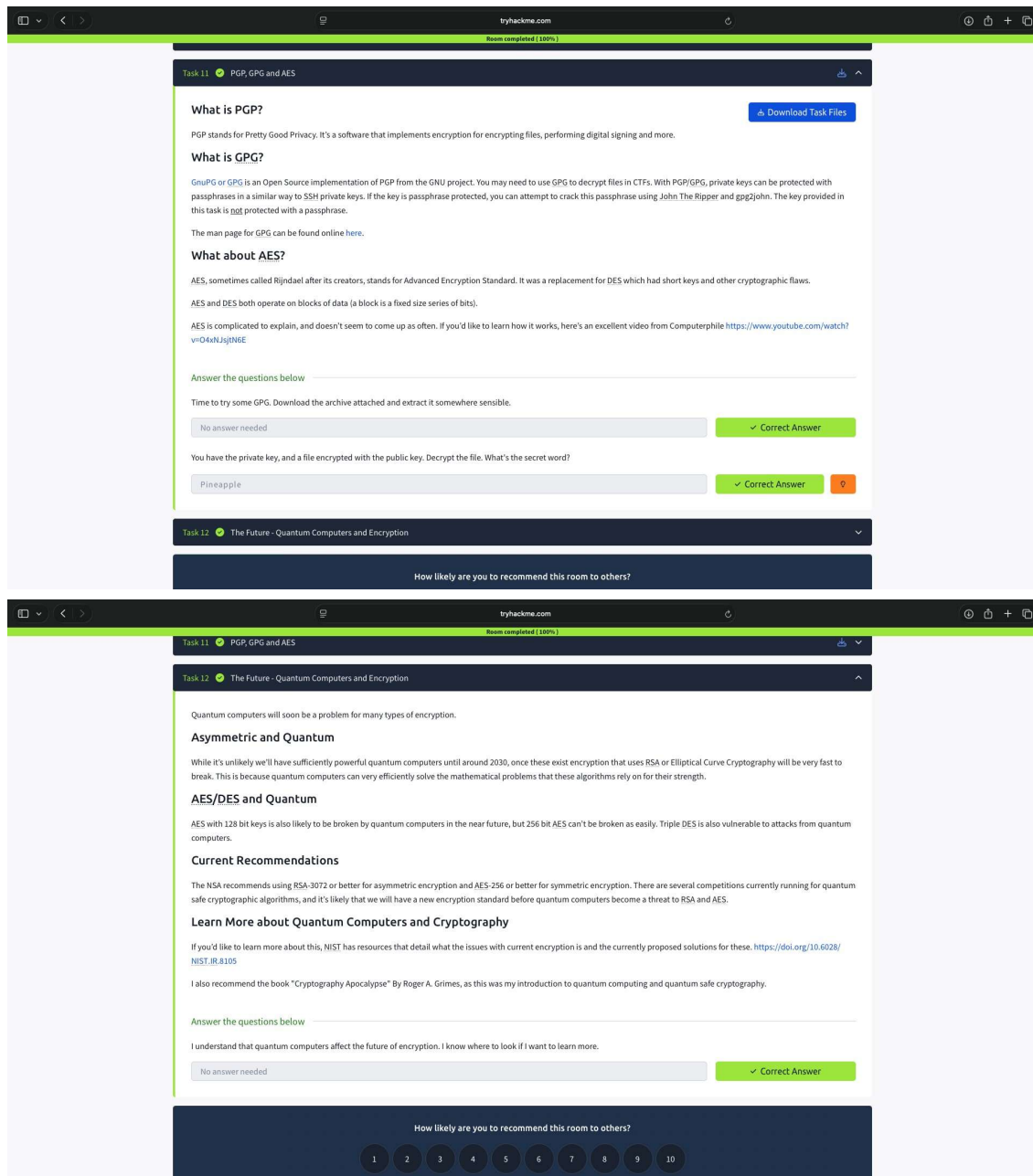
DH Key Exchange is often used alongside RSA public key cryptography, to prove the identity of the person you're talking to with digital signing. This prevents someone from attacking the connection with a man-in-the-middle attack by pretending to be Bob.

Answer the questions below

I understand how Diffie Hellman Key Exchange works at a basic level

No answer needed

✓ Correct Answer



Result

Basic cryptography concepts such as encoding, decoding, hashing, and classical ciphers were successfully performed using TryHackMe.

EXPERIMENT - 3

Performing MITM Attack Using Ettercap

Step 1: Start Environment

Open Oracle VirtualBox and start Kali Linux virtual machine.
Login to the system.

Step 2: Check Network

Open terminal and verify IP address:

```
ifconfig
```

Ensure the attacker and target systems are in the same network.

Step 3: Launch Tool

Start Ettercap in graphical mode:

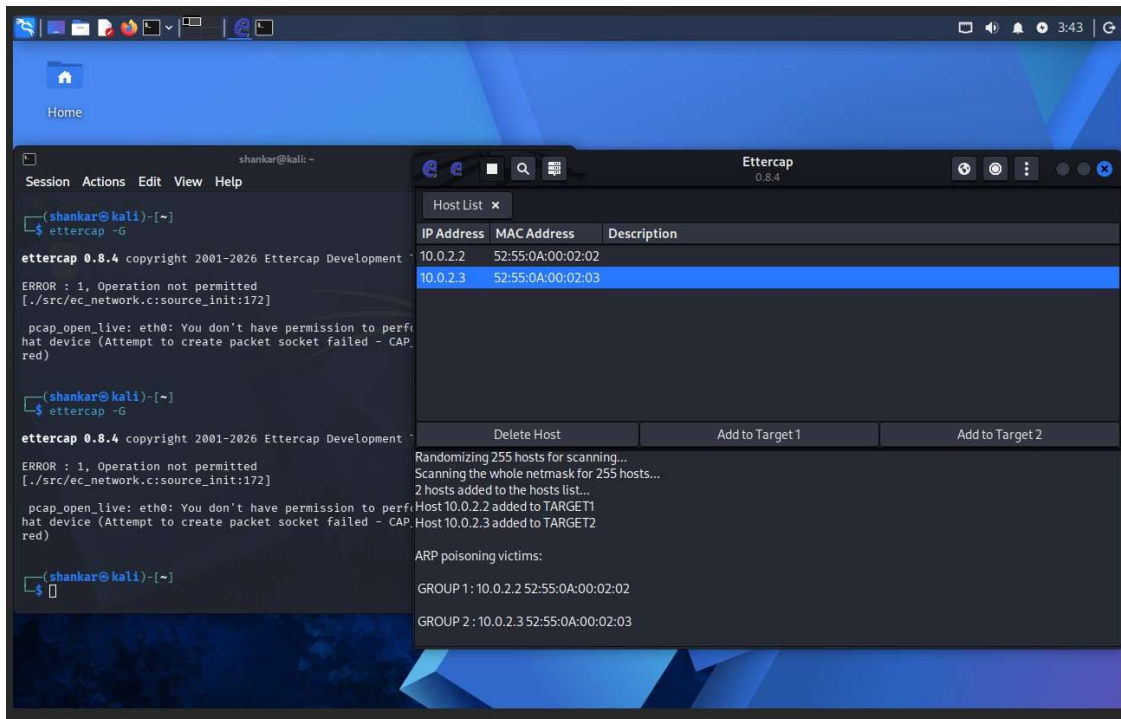
```
ettercap -G
```

Step 4: Scan and Select Targets

- Choose **Sniff** → **Unified Sniffing** and select interface
- Scan for hosts
- Identify:
 1. Target system
 2. Gateway (router)

Step 5: Perform MITM and Monitor

4. Configure MITM using ARP poisoning (conceptually)
5. Start sniffing
6. Observe intercepted network traffic in the tool



Result

The MITM attack was successfully demonstrated in a controlled environment, showing how unsecured communication can be intercepted and why encryption is important.

EXPERIMENT - 4

Hash Cracking Using John the Ripper

Step 1: Start Environment

Open Oracle VirtualBox and start Kali Linux.
Login to the system.

Step 2: Open Terminal and Verify Tool

Open terminal and check if John the Ripper is installed:

```
john --version
```

Step 3: Create Hash File

Create a file and store a password hash:

```
nano hash.txt
```

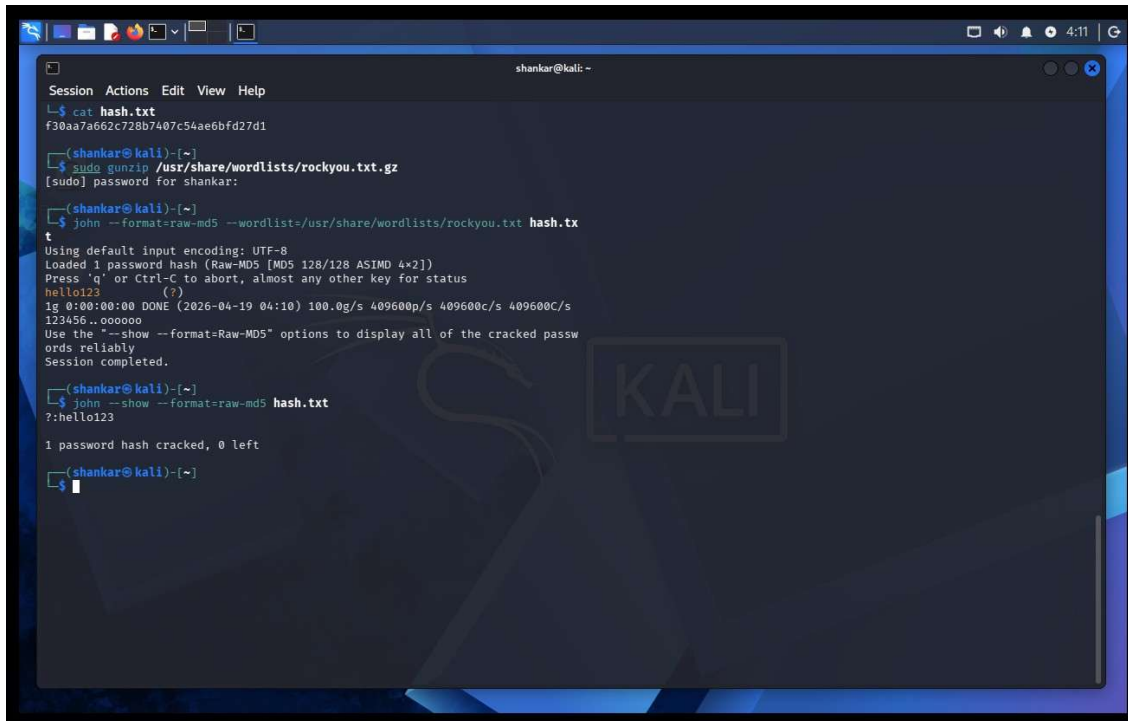
Step 4: Run Hash Cracking

```
john hash.txt
```

The tool attempts to crack the hash using default wordlists.

Step 5: View Cracked Password

```
john --show hash.txt
```



```
shankar@kali: ~  
Session Actions Edit View Help  
└─$ cat hash.txt  
f30aa7a662c728b7407c54ae6bfd27d1  
  
└─(shankar@kali)-[~]  
└─$ sudo gunzip /usr/share/wordlists/rockyou.txt.gz  
[sudo] password for shankar:  
  
└─(shankar@kali)-[~]  
└─$ john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt hash.tx  
t  
Using default input encoding: UTF-8  
Loaded 1 password hash (Raw-MD5 [MD5 128/128 ASIMD 4x2])  
Press 'q' or Ctrl-C to abort, almost any other key for status  
hello123 (r)  
1g 0:00:00:00 DONE (2026-04-19 04:10) 100.0g/s 409600p/s 409600c/s 409600C/s  
123456..oooooo  
Use the "--show --format=Raw-MD5" options to display all of the cracked pass  
words reliably  
Session completed.  
  
└─(shankar@kali)-[~]  
└─$ john --show --format=raw-md5 hash.txt  
?:hello123  
  
1 password hash cracked, 0 left  
  
└─(shankar@kali)-[~]  
└─$
```

Result

Password hash cracking was successfully demonstrated, highlighting the need for strong passwords and secure hashing techniques.

EXPERIMENT - 5

IPTables Firewall Configuration in Kali Linux

Step 1: Start Environment

Open Oracle VirtualBox and start Kali Linux virtual machine. Log in to the system.

Step 2: Open Terminal and Check IPTables

Open the terminal and verify that IPTables is installed and available in the system.

Step 3: View Existing Rules

Check the current firewall rules to understand the default configuration before making changes.

Step 4: Configure Firewall Rules

Set default policies to control incoming and outgoing traffic.

Allow necessary services such as localhost communication, SSH, HTTP, and HTTPS.

Block unwanted or suspicious IP addresses.

Step 5: Save and Verify Rules

Save the configured firewall rules and verify that they are applied correctly by reviewing the rule list.

```
Linux [Running]
shankar@kali: ~
Session Actions Edit View Help
shankar@kali:~$ iptables
iptables v1.8.11 (nf_tables): no command specified
Try 'iptables -h' or 'iptables --help' for more information.
shankar@kali:~$ iptables --version
iptables v1.8.11 (nf_tables)
shankar@kali:~$ sudo iptables -F
[sudo] password for shankar:
sudo: a password is required
shankar@kali:~$ sudo iptables -L
[sudo] password for shankar:
Chain INPUT (policy ACCEPT)
target prot opt source destination
Chain FORWARD (policy ACCEPT)
target prot opt source destination
Chain OUTPUT (policy ACCEPT)
target prot opt source destination
shankar@kali:~$ sudo iptables -P INPUT DROP
shankar@kali:~$ sudo iptables -P OUTPUT ACCEPT
shankar@kali:~$ sudo iptables -A INPUT -i lo -j ACCEPT
shankar@kali:~$ sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
shankar@kali:~$ sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
shankar@kali:~$ sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT
```

```
Linux [Running]
shankar@kali: ~

Session Actions Edit View Help
└─$ sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT
(shankar@kali)-[~]
└─$ sudo iptables -A INPUT -s 192.168.1.100 -j DROP
(shankar@kali)-[~]
└─$ sudo iptables-save>firewall_rules.txt
(shankar@kali)-[~]
└─$ sudo iptables -L -v -n
Chain INPUT (policy DROP 6 packets, 456 bytes)
  pkts bytes target     prot opt in     out     source            destination
    0     0 ACCEPT     all  --  lo      *       0.0.0.0/0         0.0.0.0/0
    0     0 ACCEPT     tcp  --  *       *       0.0.0.0/0         0.0.0.0/0          tcp dpt:22
    0     0 ACCEPT     tcp  --  *       *       0.0.0.0/0         0.0.0.0/0          tcp dpt:80
    0     0 ACCEPT     tcp  --  *       *       0.0.0.0/0         0.0.0.0/0          tcp dpt:443
    0     0 DROP      all  --  *       *       192.168.1.100     0.0.0.0/0

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
  pkts bytes target     prot opt in     out     source            destination

Chain OUTPUT (policy ACCEPT 5 packets, 380 bytes)
  pkts bytes target     prot opt in     out     source            destination

(shankar@kali)-[~]
└─$ sudo iptables -L
Chain INPUT (policy DROP)
target     prot opt source                destination
ACCEPT     all  --  anywhere              anywhere
ACCEPT     tcp  --  anywhere              anywhere          tcp dpt:ssh
ACCEPT     tcp  --  anywhere              anywhere          tcp dpt:http
ACCEPT     tcp  --  anywhere              anywhere          tcp dpt:https
DROP       all  --  192.168.1.100         anywhere

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination

(shankar@kali)-[~]
└─$ sudo iptables -S
```

```
Linux [Running]
shankar@kali: ~

Session Actions Edit View Help
└─$ sudo iptables -S
-P INPUT DROP
-P FORWARD ACCEPT
-P OUTPUT ACCEPT
-A INPUT -i lo -j ACCEPT
-A INPUT -p tcp -m tcp --dport 22 -j ACCEPT
-A INPUT -p tcp -m tcp --dport 80 -j ACCEPT
-A INPUT -p tcp -m tcp --dport 443 -j ACCEPT
-A INPUT -s 192.168.1.100/32 -j DROP
(shankar@kali)-[~]
└─$ sudo iptables -L INPUT -v -n
Chain INPUT (policy DROP 12 packets, 1260 bytes)
  pkts bytes target     prot opt in     out     source            destination
    0     0 ACCEPT     all  --  lo      *       0.0.0.0/0         0.0.0.0/0
    0     0 ACCEPT     tcp  --  *       *       0.0.0.0/0         0.0.0.0/0          tcp dpt:22
    0     0 ACCEPT     tcp  --  *       *       0.0.0.0/0         0.0.0.0/0          tcp dpt:80
    0     0 ACCEPT     tcp  --  *       *       0.0.0.0/0         0.0.0.0/0          tcp dpt:443
    0     0 DROP      all  --  *       *       192.168.1.100     0.0.0.0/0

(shankar@kali)-[~]
└─$ sudo iptables -L INPUT -n | grep DROP
Chain INPUT (policy DROP)
DROP       all  --  192.168.1.100         0.0.0.0/0

(shankar@kali)-[~]
└─$ cat firewall_rules.txt
# Generated by iptables-save v1.8.11 (nf_tables) on Sun Apr 19 04:30:53 2026
*filter
:INPUT DROP [6:456]
:FORWARD ACCEPT [0:0]
:OUTPUT ACCEPT [5:380]
-A INPUT -i lo -j ACCEPT
-A INPUT -p tcp -m tcp --dport 22 -j ACCEPT
-A INPUT -p tcp -m tcp --dport 80 -j ACCEPT
-A INPUT -p tcp -m tcp --dport 443 -j ACCEPT
-A INPUT -s 192.168.1.100/32 -j DROP
COMMIT
# Completed on Sun Apr 19 04:30:53 2026
(shankar@kali)-[~]
└─$
```

Result

Various firewall rules were implemented using IPTables, demonstrating how network traffic can be controlled to enhance system security.

EXPERIMENT - 6

Snort IDS/IPS Using TryHackMe

Step 1: Login to Platform

Open TryHackMe in a browser and log in using your account.

Step 2: Open Snort Room

Search for a Snort-related room and join it to begin the experiment.

Step 3: Start AttackBox

Launch the AttackBox, which provides a ready-to-use Linux environment for performing the tasks.

Step 4: Verify Snort Installation

Open the terminal and confirm that Snort is installed and ready to use.

Step 5: Run Snort in IDS Mode

Start Snort in detection mode so that it monitors network traffic and generates alerts for suspicious activity.

Step 6: Detect Network Threats

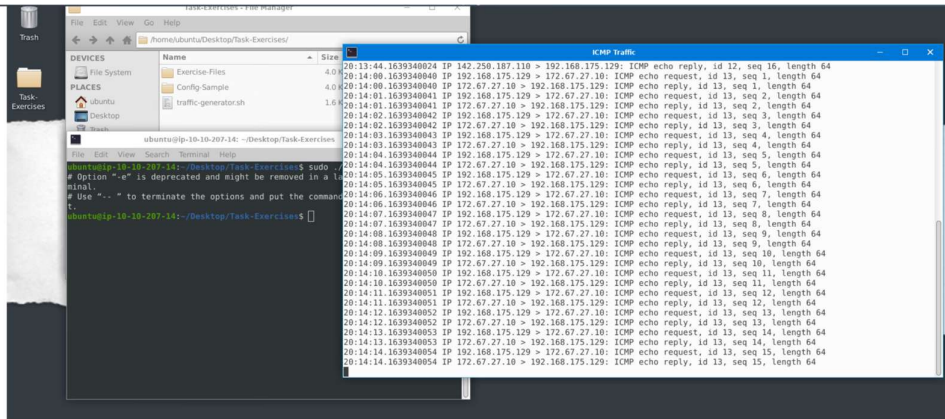
Perform the tasks given in the room and observe how Snort identifies and reports potential threats in real time.

Step 7: Configure IPS Mode

Enable prevention mode to allow Snort to block malicious traffic based on predefined rules.

Step 8: Analyze Alerts

Review the alerts generated by Snort and identify different types of network attacks.



Answer the questions below

Navigate to the Task-Exercises folder and run the command `./easy.sh` and write the output

Too Easy!

✓ Correct Answer

Task 3 Introduction to IDS/IPS

Task 4 First Interaction with Snort

Once we use a configuration file, snort has much more power: the configuration file is snort's configuration management file. Rules, plugins, detection mechanisms, default actions, and output settings are identified here. Multiple configuration files are possible for different purposes and cases, but we can only use one at runtime.

Note that Snort automatically shows the default banner and initial information about your setup when you start it. You can prevent this by using the `-q` parameter.

Parameter	Description
<code>-V / --version</code>	This parameter provides information about your instance version.
<code>-c</code>	Identifying the configuration file
<code>-T</code>	Snort's self-test parameter allows you to test your setup with this parameter.
<code>-q</code>	Quiet mode prevents Snort from displaying the default banner and initial information about your setup.

That was easy; let's move on to the next task and continue exploring snort modes.

Answer the questions below

Run the Snort instance and check the build number.

149

✓ Correct Answer



Test the current instance with `/etc/snort/snort.conf` file and check how many rules are loaded with the current build.

4151

✓ Correct Answer



Test the current instance with `/etc/snort/snortv2.conf` file and check how many rules are loaded with the current build.

1

✓ Correct Answer



Identifying interfaces, note that Snort IPS requires at least two interfaces to work. Now run the traffic-generator script as **sudo** and start **ICMP/HTTP traffic**.

```
running IPS mode

user@ubuntu$ sudo snort -c /etc/snort/snort.conf -q --daq afpacket -i eth0:eth1 -A console
Running in IPS mode

12/18-07:40:01.527100 [Drop] [**] [1:1000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 192.168.175.131 -> 192.168.1
12/18-07:40:01.552811 [Drop] [**] [1:1000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 172.217.169.142 -> 192.168.1
12/18-07:40:01.566232 [Drop] [**] [1:1000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 192.168.175.131 -> 192.168.1
12/18-07:40:02.517903 [Drop] [**] [1:1000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 192.168.1.18 -> 172.217.169.
12/18-07:40:02.550844 [Drop] [**] [1:1000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 172.217.169.142 -> 192.168.1
^C*** Caught Int-Signal
```

As you can see in the picture above, Snort blocked the packets this time. **We used the same rule with a different action (drop/reject)**. Remember, for the scope of this task, our point is the operating mode, not the rule.

Answer the questions below

Investigate the traffic with the default configuration file.

```
sudo snort -c /etc/snort/snort.conf -A full -l .
```

Execute the traffic generator script and choose "**TASK-7 Exercise**". Wait until the traffic stops, then stop the Snort instance. Now analyse the output summary and answer the question.

```
sudo ./traffic-generator.sh
```

What is the number of the detected HTTP GET methods?

✓ Correct Answer



You can practice the rest of the parameters by using the traffic-generator script.

✓ Correct Answer

Keep reading the output. How many TCP Segments are Queued?

✓ Correct Answer

Keep reading the output. How many "HTTP response headers" were extracted?

✓ Correct Answer

Investigate the **mx-1.pcap** file with the **second** configuration file.

```
sudo snort -c /etc/snort/snortv2.conf -A full -l . -r mx-1.pcap
```

What is the number of the generated alerts?

✓ Correct Answer

Investigate the **mx-2.pcap** file with the default configuration file.

```
sudo snort -c /etc/snort/snort.conf -A full -l . -r mx-2.pcap
```

What is the number of the generated alerts?

✓ Correct Answer



Keep reading the output. What is the number of the detected TCP packets?

✓ Correct Answer

Investigate the **mx-2.pcap** and **mx-3.pcap** files with the default configuration file.

```
sudo snort -c /etc/snort/snort.conf -A full -l . --pcap-list="mx-2.pcap mx-3.pcap"
```

What is the number of the generated alerts?

✓ Correct Answer

Result

Snort was successfully used as an IDS/IPS to detect and prevent network threats, demonstrating real-time network security monitoring in a controlled environment.

EXPERIMENT – 7

Performing Code Injection on Application Process Using Ptrace in Kali Linux

Step 1: Start Environment

Open Oracle VirtualBox and start Kali Linux virtual machine. Log in to the system.

Step 2: Create Target Program

Create a simple C program that runs continuously. This program will act as the target process for testing.

Step 3: Run Target Application

Execute the target program and identify its Process ID (PID), which is required to interact with the running process.

Step 4: Create Injector Program

Write a C program that uses the ptrace system call to attach to the target process and access its internal state such as registers or memory.

Step 5: Compile Programs

Compile both the target and injector programs using a C compiler so they can be executed.

Step 6: Attach to Target Process

Run the injector program and connect it to the target process using the PID. This allows control over the target process.

Step 7: Modify Execution

Change the target process behavior by altering its execution flow or memory content using ptrace.

Step 8: Observe Changes

Resume the process and observe how its behavior is modified due to the injection.

```
student@kali: ~  
Session Actions Edit View Help  
(student@kali)-[~]  
$ pgrep target  
1482  
(student@kali)-[~]  
$ sudo ./injector 1482  
[sudo] password for student:  
Attached to process 1482  
Original RIP: 7f4023b5d687  
Modified RIP:7f4023b5d689  
Detached from process  
(student@kali)-[~]  
$
```

Result

Code injection using ptrace was successfully demonstrated, showing how process execution can be controlled and highlighting the importance of system-level security and process isolation.

EXPERIMENT - 8

Privilege Escalation Using TryHackMe

Step 1: Login to Platform

Open TryHackMe and log in

Step 2: Open Room

Search for **Linux Privilege Escalation** and join the room.

Step 3: Start AttackBox

Launch the AttackBox and open the terminal.

Step 4: Perform Enumeration

Check user details, system information, and permissions to identify possible vulnerabilities.

Step 5: Identify Vulnerabilities

Look for misconfigurations such as sudo permissions, SUID files, weak file access, or cron jobs.

Step 6: Escalate Privileges

Exploit the identified vulnerability to gain higher privileges (root access).

Step 7: Verify and Complete Tasks

Confirm root access and capture required flags to complete the exercise.

TryHackMe

DashboardLearnPracticeCompete

Go Premium

1

S

← Back to all walkthroughs



Linux Privilege Escalation

Learn the fundamentals of Linux privilege escalation. From enumeration to exploitation, get hands-on with over 8 different privilege escalation techniques.

50 min

161,943

100

Start AttackBox

Save Room

5884 Recommend

Options

Room progress (30%)

Task 1 Introduction

Task 2 What is Privilege Escalation?

Task 3 Enumeration

Task 4 Automated Enumeration Tools

Task 5 Privilege Escalation: Kernel Exploits

Task 6 Privilege Escalation: Sudo

Below is a short example used to find files that have the SUID bit set. The SUID bit allows the file to run with the privilege level of the account that owns it, rather than the account which runs it. This allows for an interesting privilege escalation path, we will see in more details on task 6. The example below is given to complete the subject on the "find" command.

- `find / -perm -u+s -type f 2>/dev/null` Find files with the SUID bit, which allows us to run the file with a higher privilege level than the current user.

General Linux Commands

As we are in the Linux realm, familiarity with Linux commands, in general, will be very useful. Please spend some time getting comfortable with commands such as `find`, `locate`, `grep`, `cut`, `sort`, etc.

Answer the questions below

What is the hostname of the target system?

wade7363

✓ Correct Answer

What is the Linux kernel version of the target system?

3.13.0-24-generic

✓ Correct Answer

What Linux is this?

Ubuntu 14.04 LTS

✓ Correct Answer

What version of the Python language is installed on the system?

2.7.6

✓ Correct Answer

What vulnerability seem to affect the kernel of the target system? (Enter a CVE number)

CVE-2015-1328

✓ Correct Answer

The Kernel exploit methodology is simple;

1. Identify the kernel version
2. Search and find an exploit code for the kernel version of the target system
3. Run the exploit

Although it looks simple, please remember that a failed kernel exploit can lead to a system crash. Make sure this potential outcome is acceptable within the scope of your penetration testing engagement before attempting a kernel exploit.

Research sources:

1. Based on your findings, you can use Google to search for an existing exploit code.
2. Sources such as <https://www.cvedetails.com/> can also be useful.
3. Another alternative would be to use a script like LES (Linux Exploit Suggester) but remember that these tools can generate false positives (report a kernel vulnerability that does not affect the target system) or false negatives (not report any kernel vulnerabilities although the kernel is vulnerable).

Hints/Notes:

1. Being too specific about the kernel version when searching for exploits on Google, Exploit-db, or searchsploit
2. Be sure you understand how the exploit code works BEFORE you launch it. Some exploit codes can make changes on the operating system that would make them unsecured in further use or make irreversible changes to the system, creating problems later. Of course, these may not be great concerns within a lab or CTF environment, but these are absolute no-nos during a real penetration testing engagement.
3. Some exploits may require further interaction once they are run. Read all comments and instructions provided with the exploit code.
4. You can transfer the exploit code from your machine to the target system using the `SimpleHTTPServer` Python module and `wget` respectively.

Answer the questions below

find and use the appropriate kernel exploit to gain root privileges on the target system.

No answer needed

✓ Correct Answer



What is the content of the flag1.txt file?

THM-28392872729920

✓ Correct Answer

Result

Privilege escalation was successfully performed, demonstrating how system misconfigurations can lead to unauthorized access and highlighting the importance of secure system practices.

EXPERIMENT - 9

Windows Exploitation Using Metasploit

Step 1: Login to Platform

Open TryHackMe and log in.

Step 2: Open Room

Search for a **Metasploit / Windows Exploitation** room and join it.

Step 3: Start AttackBox

Launch the AttackBox and open the terminal.

Step 4: Launch Metasploit

Start the Metasploit Framework console to begin the exploitation process.

Step 5: Scan Target System

Identify the target system and perform basic scanning to find vulnerabilities.

Step 6: Select Exploit

Choose an appropriate Windows exploit module based on the identified vulnerability.

Step 7: Configure Parameters

Set target details and required options such as payload and connection settings.

Step 8: Execute Exploit

Run the exploit to gain access to the target system.

Step 9: Post-Exploitation

Verify access, check system information, and perform basic enumeration.

Step 10: Complete Tasks

Capture required flags and mark all tasks as completed.

Result

Windows vulnerabilities were successfully exploited using the Metasploit Framework, demonstrating practical knowledge of system exploitation and post-exploitation techniques.

EXPERIMENT – 10

Wireless Audit Using Aircrack-ng

Step 1: Start Environment

Open Oracle VirtualBox and start Kali Linux. Connect the external Wi-Fi adapter.

Step 2: Verify Wireless Interface

Check the available wireless interface and ensure it is detected properly.

Step 3: Enable Monitor Mode

Switch the wireless adapter to monitor mode to capture network packets.

Step 4: Scan Wireless Networks

Scan nearby Wi-Fi networks and identify the target network details such as BSSID and channel.

Step 5: Capture Handshake

Capture the WPA/WPA2 handshake by monitoring the target network traffic.

Step 6: Perform Password Cracking

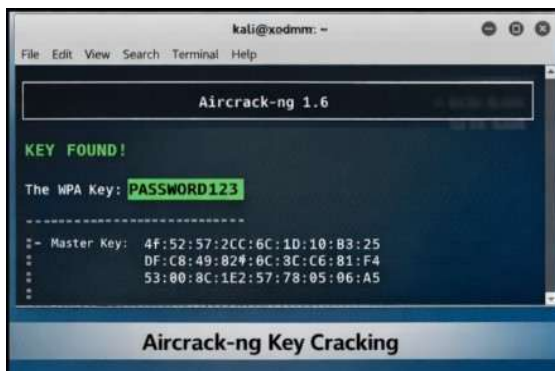
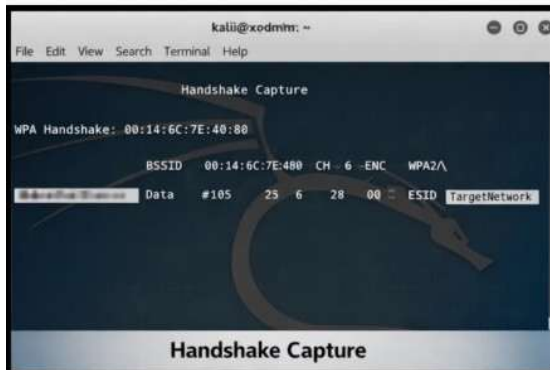
Use a wordlist to attempt password recovery from the captured handshake.

Step 7: Stop Monitoring

Disable monitor mode after completing the analysis.

Output

WPA/WPA2 handshake is captured and the password cracking process is performed successfully.



Result

Wireless auditing was successfully demonstrated using Aircrack-ng, highlighting the importance of strong passwords and secure wireless configurations.

EXPERIMENT-11

IP Spoofing Using GitHub Tool in Kali Linux

Step 1: Start Environment

Open Oracle VirtualBox and start Kali Linux.

Step 2: Prepare Tool

Download or clone an IP spoofing tool from GitHub and install required dependencies.

Step 3: Configure Parameters

Set the target IP address, spoofed source IP, and network interface in the tool.

Step 4: Execute Tool

Run the tool to send packets with a forged source IP address.

Step 5: Monitor Traffic

Observe the network traffic using monitoring tools to verify spoofed packets.

Output

Packets are transmitted with modified (spoofed) source IP addresses and reflected in network traffic.

```
shankar@kali: ~  
Session Actions Edit View Help  
(shankar@kali)-[~]  
$ sudo tcpdump -i any -n src 1.2.3.4  
[sudo] password for shankar:  
tcpdump: WARNING: any: That device doesn't support promiscuous mode  
(Promiscuous mode not supported on the "any" device)  
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode  
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes  
08:21:35.152413 lo In IP 1.2.3.4 > 127.0.0.1: ICMP echo request, id 0, seq 0, length 8  
08:21:35.152905 lo In IP 1.2.3.4 > 127.0.0.1: ICMP echo request, id 0, seq 0, length 8  
08:21:35.153544 lo In IP 1.2.3.4 > 127.0.0.1: ICMP echo request, id 0, seq 0, length 8  
08:21:35.153915 lo In IP 1.2.3.4 > 127.0.0.1: ICMP echo request, id 0, seq 0, length 8  
08:21:35.155276 lo In IP 1.2.3.4 > 127.0.0.1: ICMP echo request, id 0, seq 0, length 8  
08:21:35.155669 lo In IP 1.2.3.4 > 127.0.0.1: ICMP echo request, id 0, seq 0, length 8  
08:21:35.156023 lo In IP 1.2.3.4 > 127.0.0.1: ICMP echo request, id 0, seq 0, length 8  
08:21:35.156371 lo In IP 1.2.3.4 > 127.0.0.1: ICMP echo request, id 0, seq 0, length 8  
08:21:35.156714 lo In IP 1.2.3.4 > 127.0.0.1: ICMP echo request, id 0, seq 0, length 8  
08:21:35.157048 lo In IP 1.2.3.4 > 127.0.0.1: ICMP echo request, id 0, seq 0, length 8
```

Result

IP spoofing was successfully demonstrated, showing how packet headers can be manipulated and emphasizing the need for strong network security measures.

EXPERIMENT - 12

Camera Phishing Study in Kali Linux

Step 1: Start Environment

Open Oracle VirtualBox and start Kali Linux.

Step 2: Study Tool

Refer to an open-source repository on GitHub and understand how camera phishing works.

Step 3: Analyze Workflow

Study how fake permission prompts are created and how users are tricked into granting camera access.

Step 4: Simulated Demonstration

Perform a consent-based simulation and observe camera permission requests and user responses.

Step 5: Identify Risks and Prevention

Understand risks like privacy leakage and learn prevention methods such as permission control and user awareness.

Output

Understanding of camera phishing techniques and associated privacy risks.

Camera Phishing

A page to take pictures from the front and back camera

This program requires php, ngrok and python! Please install these prerequisites before running

requires

- Install Ngrok
 - Widnows: <https://ngrok.com/download/windows>
 - Linux: <https://ngrok.com/download/linux>
 - Termux: <https://github.com/Yisus7u7/termux-ngrok>
- Install php
 - Windows: <https://www.php.net/downloads.php>
 - Linux: sudo apt install php
 - Termux: pkg install php
- Install Python (py)
 - Windows: [Download](#)
 - Linux: sudo apt install python
 - Termux: pkg install python

ScreenShot



After installing ngrok, enter the site and register, get your token and enter the token with this command: `ngrok config`

`add-authtoken <token>`

Now run the program:

```
git clone https://github.com/Mr-Spect3r/CamPhish
cd CamPhish
python main.py
```

Result

Camera phishing concepts were successfully studied, highlighting the importance of user awareness and secure permission handling.

EXPERIMENT - 13

Phishing Attack Study Using Z-Phisher (Simple Steps)

Step 1: Start Environment

Open Oracle VirtualBox and start Kali Linux.

Step 2: Study Tool

Refer to the Z-Phisher framework on GitHub and understand its features.

Step 3: Analyze Workflow

Study how phishing pages imitate real websites and trick users into entering credentials.

Step 4: Simulated Demonstration

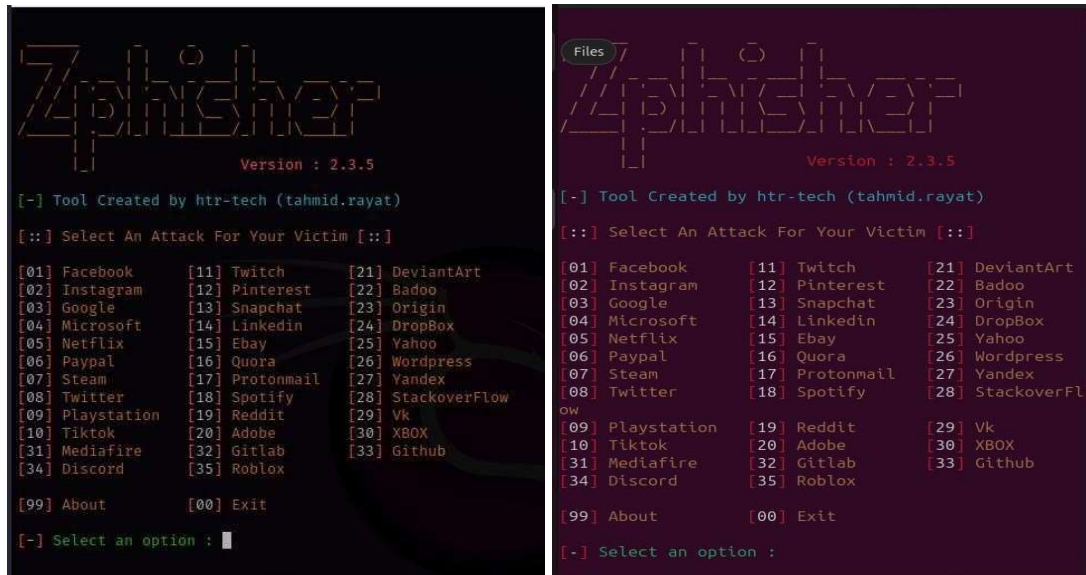
Observe phishing page behavior using a safe, consent-based simulation with dummy data.

Step 5: Identify Risks and Prevention

Understand risks like credential theft and learn prevention methods such as user awareness, HTTPS verification, and two-factor authentication.

Output

Understanding of phishing techniques and social-engineering risks.



Result

Phishing concepts were successfully studied, emphasizing user awareness and security practices.

EXPERIMENT - 14

Brute Force Attack Study in Kali Linux

Step 1: Start Environment

Open Oracle VirtualBox and start Kali Linux.

Step 2: Setup Test Environment

Use a local test account with dummy credentials in an isolated environment.

Step 3: Study Attack Concept

Understand how repeated login attempts are used to guess passwords.

Step 4: Simulated Demonstration

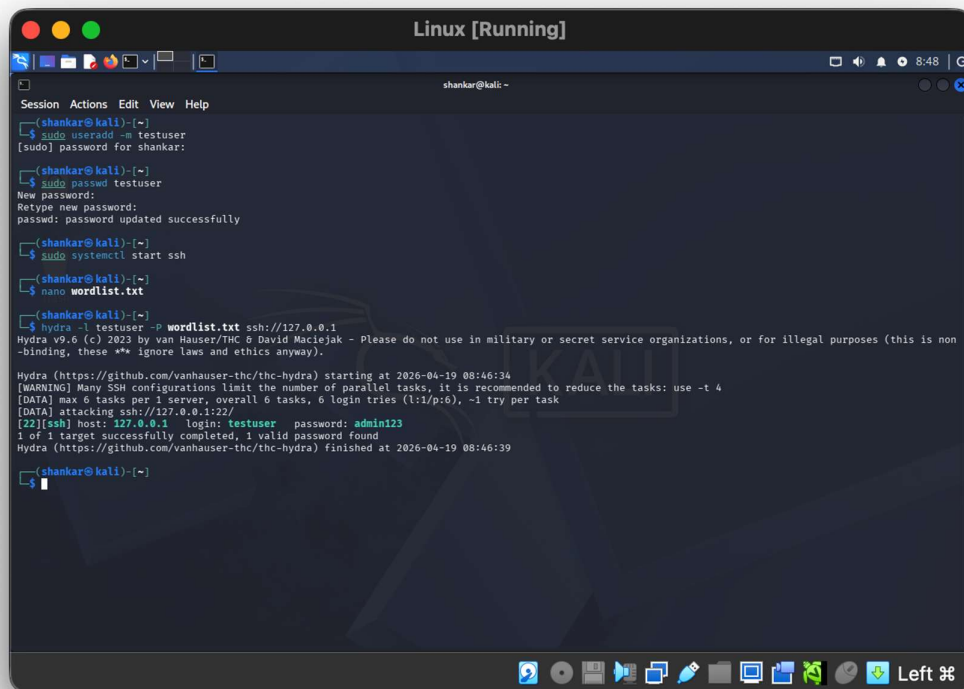
Observe multiple authentication attempts and system responses in a controlled setup.

Step 5: Analyze Risks and Prevention

Identify risks of weak passwords and study prevention methods like strong passwords, account lockout, and multi-factor authentication.

Output

Simulated login attempts demonstrate how weak passwords can be guessed.



The screenshot shows a Kali Linux terminal window titled "Linux [Running]". The user "shankar" is logged in. The terminal history includes the following commands and output:

```
(shankar@kali)~$ sudo useradd -m testuser
[sudo] password for shankar:
(shankar@kali)~$ sudo passwd testuser
New password:
Retype new password:
passwd: password updated successfully
(shankar@kali)~$ sudo systemctl start ssh
(shankar@kali)~$ nano wordlist.txt
(shankar@kali)~$ hydra -l testuser -P wordlist.txt ssh://127.0.0.1
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-04-19 08:46:34
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 6 tasks per 1 server, overall 6 tasks, 6 login tries (l:1/p:6), ~1 try per task
[22][ssh] host: 127.0.0.1  login: testuser  password: admin123
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-04-19 08:46:39
(shankar@kali)~$
```

The output of the Hydra command shows a successful brute force attack on the SSH service of the local host (127.0.0.1) using the username "testuser" and the password "admin123".

Result

Brute force attack concepts were successfully studied, highlighting the importance of strong authentication mechanisms.